

Universidad Internacional de Valencia

Master Universitario en Robotica y Automatizacion de Procesos Industriales

Sistemas Roboticos Moviles

Actividad 2

Simulacion de warehouse con Pioneer, Bill, herramientas y
SLAM en CoppeliaSim/Lua

Entrega practica de Warehouse R1-B1-T1/T2

Autor

Jorge Luis Mayorga Taborda

Profesor

Jose I. Iniguez

jiniguez@professor.universidadviu.com

Entrega

25 de mayo de 2026, antes de las 23:59

Índice general

Resumen ejecutivo	5
1. Introduccion	6
1.1. Objetivos tecnicos	6
1.2. Criterio de honestidad academica	6
1.3. Vista general del resultado	7
2. Escenario CoppeliaSim	8
2.1. Elementos del warehouse	8
2.2. Bill como actor movil	9
2.3. Cola de tareas	9
2.4. Piezas visuales	9
3. Modelo del robot y control	10
3.1. Seguimiento de objetivo	10
3.2. Modos de control evaluados	11
4. Evitacion de obstaculos y planificacion local	13
4.1. Campo reactivo de obstaculos	13
4.2. Visualizacion de sensores	13
4.3. Waypoint de planificacion SLAM	13
4.4. Lectura operacional	14
5. Estimacion y SLAM	15
5.1. Prediccion de pose	15
5.2. Correccion Kalman	15
5.3. GMapping reducido	16
5.4. Hector reducido	16
5.5. Cartographer reducido	16
6. Resultados experimentales	17
6.1. Validacion funcional de la escena	17

6.2. Desempeno del estimador Kalman-landmark	18
6.3. Comparacion de algoritmos SLAM	18
6.4. Lectura de resultados	19
7. Validacion y trazabilidad	21
7.1. Checklist automatizado	21
7.2. Timeline de la mision	22
7.3. Senales exportadas	22
7.4. Reproducibilidad	23
8. Conclusiones	24
Bibliografia	25
A. Anexos	26
A.1. Archivos principales	26
A.2. Parametros relevantes	27
A.3. Resumen de benchmark de control	27
A.4. Notas de limitacion	28

Índice de figuras

1.1. Vista superior reconstruida de la celda Sim_T2_Phase1_Basic: Bill trabaja entre dos mesas, T1/T2 se almacenan en shelves opuestos y R1 planifica rutas con obstaculos y carga.	7
2.1. Maquina de estados resumida del controlador R1 para entrega, trabajo de Bill, retorno de herramienta y carga.	9
3.1. Comparacion de modos de control exportada desde CoppeliaSim. Todos completan la tarea; PI obtiene el mejor score global en los pesos definidos.	12
5.1. Pipeline usado por la escena: odometria y sensores alimentan estimacion, mapa, planificacion local y control de R1.	15
6.1. Desempeno del estimador Kalman-landmark: error de pose, covarianza publicada, landmarks y actualizaciones.	18
6.2. Comparacion grafica de los cuatro modos SLAM reducidos: precision, mapa, actualizaciones y bateria.	19
7.1. Timeline exportado de la simulacion: estados de tarea, movimiento, planner y acciones de Bill durante la mision.	22
7.2. Resumen visual de la corrida base: pose, error, bateria, obstaculos, planner y estados principales.	23
A.1. Metricas complementarias de control: tiempos, trayectoria, suavidad, bateria, riesgo y error de pose.	28

Resumen ejecutivo

Este documento presenta la Actividad 2 como una simulación reproducible en CoppeliaSim/Lua. La escena final `Sim_T2_Phase1_Basic.ttt` modela un warehouse con un robot Pioneer P3DX identificado como R1, una persona móvil B1 (Bill), dos estaciones de herramienta T1/T2, dos mesas de trabajo, obstáculos, una estación de carga C1 y una cola de tareas de entrega/retorno de piezas.

La contribución principal no es una escena decorativa, sino un ciclo completo de robótica móvil: lectura de sensores tipo LiDAR/proximidad, estimación de pose, construcción de mapa, planificación local, evitación de obstáculos, control de velocidades diferenciales, gestión de batería y coordinación con una persona móvil. La validación automatizada confirma que R1 completa cuatro tareas:

- `task1:{T1 -> B1@WS1}`: R1 recoge T1 en el rack y la entrega a Bill.
- `task2:{B1@WS1(T1) -> Rack_T1}`: Bill trabaja la pieza y R1 la devuelve al shelf.
- `task3:{T2 -> B1@WS2}`: Bill se desplaza a la segunda mesa y solicita T2.
- `task4:{B1@WS2(T2) -> Rack_T2}`: R1 recoge la pieza trabajada y la almacena.

El benchmark compara cuatro estimadores/mappers reducidos, implementados dentro de CoppeliaSim y alimentados por las mismas señales: `KALMAN_LANDMARK`, `GMAPPING_GRID`, `HECTOR_GRID_MATCHING` y `CARTOGRAPHER_SUBMAP`. Se declara explícitamente que no se ejecutan los paquetes ROS originales; se trata de implementaciones académicas reducidas, inspiradas en los métodos clásicos de la literatura.

El resultado final de validación de la escena principal fue satisfactorio: cuatro tareas completadas, retorno a carga, 16 sensores activos, rayos de sensor visualizados, evitación reactiva activa, SLAM activo, planificación por waypoint SLAM y error final de pose acotado a 0,031 m. En la comparación de SLAM, el modo Kalman-landmark obtiene el menor RMSE de posición (0,031.6 m), mientras que Cartographer-submap produce el mapa más rico en estructura temporal con 10 submaps y 1 cierre de ciclo simplificado.

1

Introduccion

La Actividad 2 solicita programar un robot terrestre Pioneer en CoppeliaSim/Lua y validar su navegacion en una celda robotizada. La solucion desarrollada lleva el ejercicio a un caso de fabrica: un robot R1 debe asistir a una persona movil B1 (Bill), entregar herramientas/piezas en dos mesas de trabajo, recogerlas una vez procesadas, almacenarlas de nuevo en el shelf correcto y regresar a una estacion de carga si la mision queda completada.

El escenario se diseno de forma incremental. Primero se creo una version basica con un solo robot, una sola persona y dos herramientas. Esta decision reduce el riesgo: permite validar estimacion, sensores, evitacion, rutas y bateria antes de escalar a varios Bill y varios robots. La escena final de esta fase se denomina:

`actividad2/coppeliasim/Sim_T2_Phase1_Basic.ttt`

1.1 Objetivos tecnicos

- Modelar una celda tipo warehouse con dos workstations, dos tool shelves, obstaculos y estacion de carga.
- Programar en Lua el flujo de tareas R1–B1–T1/T2 con una cola de trabajo clara y trazable.
- Mantener a Bill como persona movil: camina entre estaciones, se orienta en la direccion del movimiento y trabaja frente a la mesa.
- Implementar evitacion reactiva de obstaculos usando sensores de proximidad del Pioneer visualizados como rayos tipo LiDAR.
- Incorporar estimacion de pose y mapeo: Kalman-landmark, grilla tipo GMapping, scan matching tipo Hector y submaps tipo Cartographer.
- Exportar datos de simulacion a CSV/JSON y generar plots para analizar desempeno de control, seguridad, bateria y SLAM.

1.2 Criterio de honestidad academica

El informe diferencia entre algoritmos clasicos reales y aproximaciones reducidas de simulacion. El filtro Kalman implementa un ciclo de prediccion y correccion sobre la pose y landmarks compactos. Los modos GMapping, Hector y Cartographer no ejecutan los paquetes ROS originales; reproducen ideas centrales de esos metodos dentro de CoppeliaSim: grilla de ocupacion, scan matching local, submaps y

cierre de ciclo simplificado. Esta distincion es importante para defender el trabajo con rigor y evitar presentar una simulacion educativa como un stack industrial completo.

1.3 Vista general del resultado

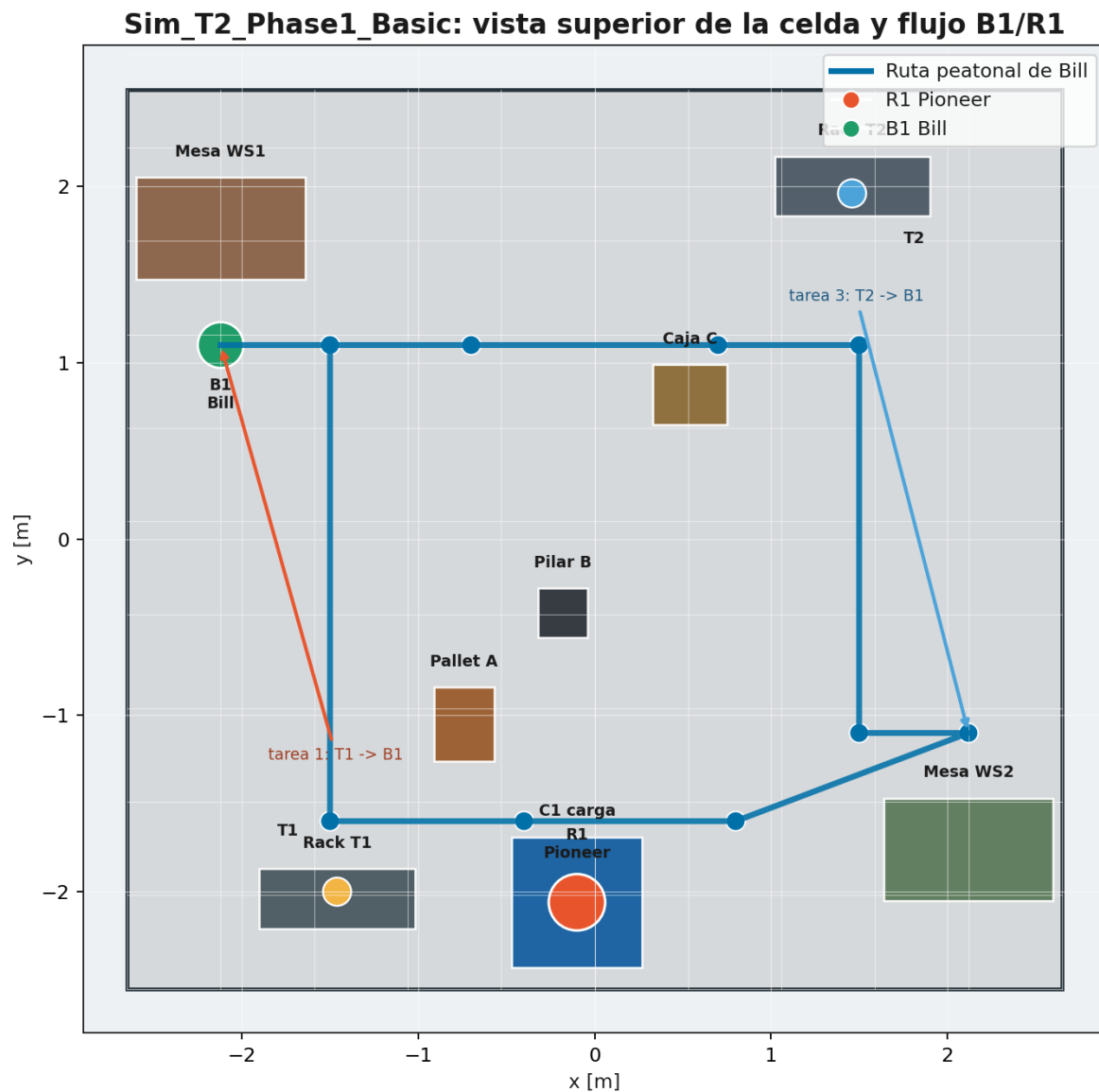


Figura 1.1: Vista superior reconstruida de la celda Sim_T2_Phase1_Basic: Bill trabaja entre dos mesas, T1/T2 se almacenan en shelves opuestos y R1 planifica rutas con obstaculos y carga.

2

Escenario Coppeliasim

La escena se construyo automaticamente con el script `actividad2/coppeliastim/build_phase1_basic_scene.py`. Este generador carga la escena base de la asignatura, elimina restos de pruebas anteriores, crea la celda warehouse, ubica a R1 y B1, agrega shelves, mesas, obstaculos, piezas visuales y embebe los controladores Lua finales. El mismo script ejecuta una validacion headless y guarda los resultados en JSON.

2.1 Elementos del warehouse

Elemento	Alias en escena	Funcion
Robot movil	/PioneerP3DX	R1 ejecuta la cola de tareas, navega, evita obstaculos, transporta herramientas y vuelve a carga.
Persona movil	/B1	Bill trabaja en dos mesas; se mueve entre WS1 y WS2 y solicita herramientas.
Herramienta/pieza 1	/T1	Pieza mecanizada visual, entregada en WS1 y devuelta a Rack_T1.
Herramienta/pieza 2	/T2	Pieza tipo placa electronica/compuesta, entregada en WS2 y devuelta a Rack_T2.
Estacion de carga	/C1	Punto final de retorno y recarga del robot.
Mesas de trabajo	/P1_WorkTable_1, /P1_WorkTable_2	Superficies donde Bill toma la pieza, la trabaja y la devuelve a R1.
Shelves	/Rack_T1, /Rack_T2	Estaciones de almacenamiento opuestas para T1 y T2.
Obstaculos	Pallet, pilar y crate	Elementos detectables para activar evitacion reactiva y mapeo.

2.2 Bill como actor movil

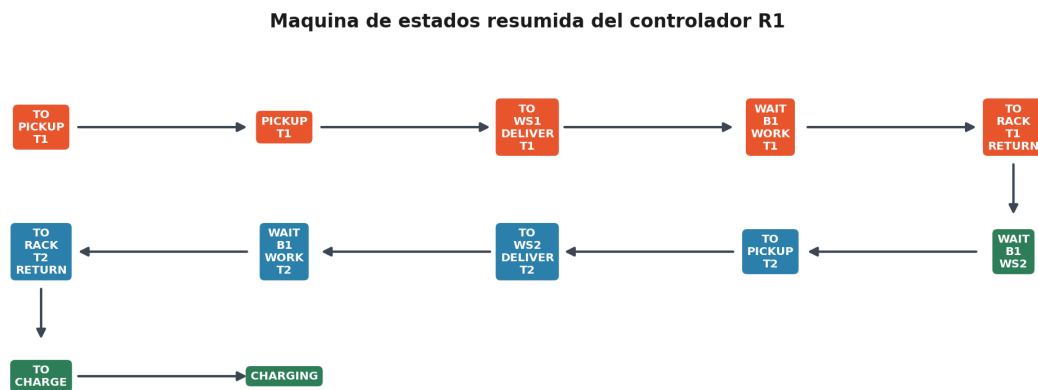
B1 no es un obstaculo estatico: representa una persona que se desplaza entre estaciones de trabajo. El script `phase1_b1_walking_manager.lua` define las estaciones WS1/WS2 como datos, publica la estacion actual, informa que herramienta necesita y anima las acciones de tomar, trabajar y entregar. La orientacion de Bill se calcula con:

$$\theta_B = \text{atan2}(y_{k+1} - y_k, x_{k+1} - x_k)$$

De esta forma Bill camina orientado hacia la direccion de movimiento. Cuando esta en una mesa, su yaw se orienta hacia la superficie de trabajo para evitar que parezca trabajar de lado o de espaldas.

2.3 Cola de tareas

La cola de tareas se representa mediante senales internas de CoppeliaSim y una maquina de estados en el controlador R1. El flujo completo se resume en la Figura 2.1. Las tareas no se disparan todas al mismo tiempo: R1 espera a que Bill termine T1 y se desplace a WS2 antes de iniciar T2.



Las transiciones de navegacion se disparan por tolerancia de llegada; las de manipulacion por temporizador y senales de Bill.

Figura 2.1: Maquina de estados resumida del controlador R1 para entrega, trabajo de Bill, retorno de herramienta y carga.

2.4 Piezas visuales

Para evitar cubos genericos, T1 y T2 se construyen como conjuntos de primitivas: T1 parece una pieza mecanizada con placa, nervios, boss central y pernos; T2 parece una placa compuesta/electronica con railes, conector y capacitor. Esto mejora la lectura visual y permite que las capturas del escenario comuniquen que se estan manipulando herramientas/piezas reales, no marcadores abstractos.

3

Modelo del robot y control

El Pioneer 3-DX se modela como robot diferencial. El controlador calcula una velocidad lineal v y una velocidad angular ω , y las transforma a velocidades de rueda izquierda y derecha:

$$v_L = v - \frac{L}{2}\omega, \quad v_R = v + \frac{L}{2}\omega$$

Si r es el radio de rueda, las consignas enviadas a los motores de CoppeliaSim son:

$$\omega_L = \frac{v_L}{r}, \quad \omega_R = \frac{v_R}{r}$$

En la implementación se usan $r = 0,097.5$ m y $L = 0,331$ m. Las velocidades se saturan para proteger la simulación y mantener trayectorias reproducibles.

3.1 Seguimiento de objetivo

Para navegar hacia un rack, mesa o cargador, se calcula primero la posición del objetivo en coordenadas globales y luego el error respecto a la pose estimada del robot:

$$e_x = x_g - \hat{x}, \quad e_y = y_g - \hat{y}, \quad d = \sqrt{e_x^2 + e_y^2}$$

El rumbo deseado es:

$$\theta_g = \text{atan2}(e_y, e_x)$$

y el error angular usado por el controlador es:

$$e_\theta = \text{wrap}(\theta_g - \hat{\theta})$$

La consigna base se corrige con un término lateral y con una contribución de evitación de obstáculos. En forma compacta:

$$v = f_d(d - d_f) \cos(e_\theta), \quad \omega = f_\theta(e_\theta) + f_{lat}(e_y^{body}) + f_{obs}$$

donde d_f es la distancia de seguimiento y f_{obs} resume el giro repulsivo derivado de sensores.

3.2 Modos de control evaluados

El benchmark de control se ejecuto con cinco modos: P, PI, PID, LQR y NMPC. La parte principal defendible academicamente en esta entrega es P/PI/PID junto con la estimacion y evitacion de obstaculos. Los modos LQR y NMPC se conservan en los logs como aproximaciones reducidas dentro del script, utiles para comparar suavidad y consumo, pero no se presentan como un solver LQR/NMPC industrial.

Modo	Idea	Uso en la escena
P	Control proporcional sobre distancia y heading.	Simple, estable y facil de explicar, pero con mayor tiempo activo y ruta mas larga.
PI	Agrega integral de distancia/heading.	Fue el mejor por el score ponderado de la comparacion de controladores.
PID	Incluye derivadas para amortiguar cambios.	Buen compromiso general; se uso como modo base en varias exportaciones SLAM.
LQR	Realimentacion lineal reducida de error lateral y heading.	Se conserva como aproximacion academica ligera, no como diseno completo por Riccati.
NMPC	Penalizacion heuristica de riesgo y heading.	No usa solver no lineal; se reporta como modo predictivo reducido para comparacion.

Phase 1 Basic - Comparacion de controladores en la misma secuencia T1/T2

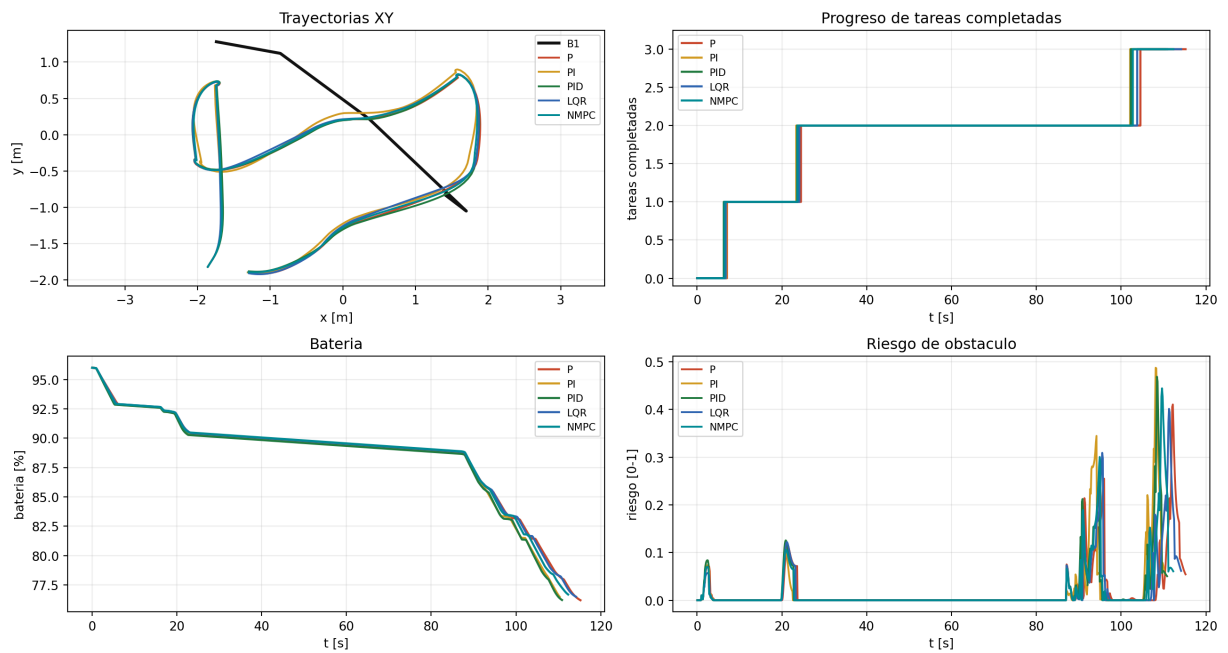


Figura 3.1: Comparacion de modos de control exportada desde CoppeliaSim. Todos completan la tarea; PI obtiene el mejor score global en los pesos definidos.

4

Evitacion de obstaculos y planificacion local

La navegacion combina dos capas. La primera es reactiva: los sensores de proximidad generan reduccion de velocidad y giro de escape si un obstaculo se aproxima al frente o a los laterales. La segunda es deliberativa local: el mapa SLAM se consulta para decidir si conviene desviarse mediante un waypoint intermedio antes de volver al objetivo.

4.1 Campo reactivo de obstaculos

Para cada sensor con deteccion positiva se calcula una influencia normalizada:

$$\alpha_i = \begin{cases} \left(\frac{d_0 - d_i}{d_0} \right)^2, & 0 < d_i < d_0 \\ 0, & d_i \geq d_0 \end{cases}$$

donde d_i es la distancia medida y d_0 el rango de influencia. La contribucion angular se aproxima con el lado del sensor:

$$\omega_{obs} = - \sum_i \text{sgn}(y_i) k_{rep} \alpha_i w_i$$

El factor w_i aumenta el peso de sensores frontales y reduce el de sensores traseros. Si un obstaculo esta dentro de una distancia critica frontal, el robot entra en modo AVOIDING, reduce v y gira en el sentido de menor riesgo.

4.2 Visualizacion de sensores

Los 16 sensores ultrasonicos del Pioneer se visualizan como rayos. Cuando no hay impacto, el rayo se dibuja hasta el rango nominal; cuando hay obstaculo, se dibuja hasta el punto detectado. Esta visualizacion cumple una funcion doble: permite validar que la percepcion local existe y ayuda a explicar por que el robot modifica su trayectoria ante pallets, pilares o cajas.

4.3 Waypoint de planificacion SLAM

El planificador no ejecuta A* global en esta fase. Usa una regla simple y trazable: si una landmark o celda ocupada cae demasiado cerca del segmento directo entre la pose estimada y el objetivo, se generan dos candidatos laterales a distancia d_{off} . Se elige el candidato con menor coste:

$$J(q) = \|q - \hat{x}_{R1}\| + \|x_g - q\| + \lambda C_{obs}(q)$$

donde C_{obs} penaliza candidatos cercanos a obstaculos mapeados. Si se activa este desvio, la senal `phase1PlannerMode` publica `SLAM_WAYPOINT`; al alcanzar el waypoint, el sistema vuelve a ruta directa.

4.4 Lectura operacional

Esta arquitectura es deliberadamente KISS: no pretende resolver planificacion global optima, sino lograr un comportamiento robusto, explicable y suficiente para la celda pequena. El robot no necesita reconstruir todo el mundo antes de moverse; actualiza el mapa mientras avanza y usa esa informacion para evitar bloqueos locales.

5

Estimacion y SLAM

El estimador no “une puntos” de forma geometrica. Ejecuta un ciclo clasico: predice la pose con odometria diferencial, corrige con mediciones ruidosas y usa esa pose corregida para transformar lecturas de sensor a coordenadas del warehouse. A partir de esas detecciones se actualiza un mapa compacto de landmarks o una grilla de ocupacion, segun el algoritmo seleccionado.

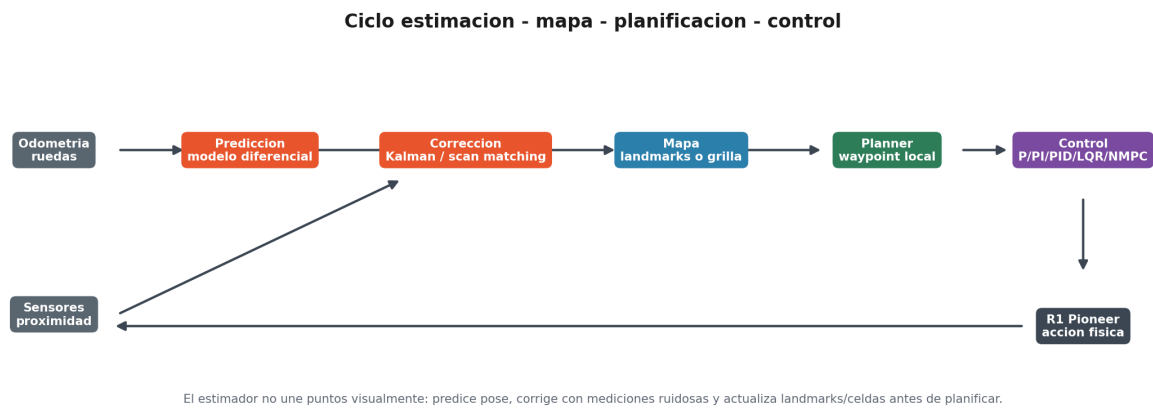


Figura 5.1: Pipeline usado por la escena: odometria y sensores alimentan estimacion, mapa, planificacion local y control de R1.

5.1 Prediccion de pose

Con velocidades odometricas v_k, ω_k , el modelo diferencial discreto es:

$$\begin{aligned}\hat{x}_k^- &= \hat{x}_{k-1}^+ + v_k \cos(\hat{\theta}_{k-1}^+) \Delta t \\ \hat{y}_k^- &= \hat{y}_{k-1}^+ + v_k \sin(\hat{\theta}_{k-1}^+) \Delta t \\ \hat{\theta}_k^- &= \text{wrap}(\hat{\theta}_{k-1}^+ + \omega_k \Delta t)\end{aligned}$$

La covarianza se propaga con el Jacobiano G_k :

$$P_k^- = G_k P_{k-1}^+ G_k^\top + Q_k$$

5.2 Correccion Kalman

La correccion usa la forma clasica del filtro de Kalman [1]:

$$K_k = P_k^- H_k^\top (H_k P_k^- H_k^\top + R_k)^{-1}$$

$$\hat{x}_k^+ = \hat{x}_k^- + K_k(z_k - H_k \hat{x}_k^-), \quad P_k^+ = (I - K_k H_k) P_k^-$$

En el modo KALMAN_LANDMARK, las detecciones de obstaculo se asocian por cercania. Si una lectura cae dentro del radio de asociacion de una landmark, se actualiza esa landmark con una ganancia escalar; si no, se crea una landmark nueva hasta un maximo configurado.

5.3 GMapping reducido

GMapping se basa en filtros de particulas Rao-Blackwellizados y mapas de grilla [2]. En esta simulacion no se ejecuta `slam_gmapping`; se implementa una version reducida de grilla de ocupacion con log-odds. Para cada rayo, las celdas atravesadas se actualizan como libres y la celda de impacto como ocupada:

$$L_t(c) = \text{clip} \left(L_{t-1}(c) + \begin{cases} l_{occ}, & c = c_{hit} \\ l_{free}, & c \in ray \end{cases} \right)$$

La grilla resultante se usa tanto para publicar telemetria como para alimentar el planner local.

5.4 Hector reducido

Hector SLAM destaca por scan matching contra una grilla sin depender fuertemente de odometria [3]. En esta escena se implementa una busqueda local discreta alrededor de la pose estimada. Para cada candidato (x, y, θ) se proyectan los puntos del scan y se evalua el score contra la grilla:

$$s(x, y, \theta) = \frac{1}{N} \sum_{i=1}^N L(x + r_i \cos(\theta + \beta_i), y + r_i \sin(\theta + \beta_i))$$

Si el mejor score supera un umbral, la pose se corrige parcialmente hacia ese candidato. Por eso el metodo se reporta como HECTOR_GRID_MATCHING, no como Hector ROS completo.

5.5 Cartographer reducido

Google Cartographer organiza el mapeo en submaps y cierres de ciclo para reducir deriva acumulada [4]. La version usada aqui mantiene submaps temporales, crea nuevos submaps por distancia o periodo y aplica un cierre de ciclo simplificado cuando la pose vuelve cerca de un submap antiguo con score suficiente. En el benchmark produjo 10 submaps y 1 loop closure.

6

Resultados experimentales

Los resultados se obtuvieron ejecutando CoppeliaSim de forma automatizada desde Python. Cada corrida exporta senales de la simulacion a CSV/JSON y luego los scripts de plot generan las graficas del informe. Esto permite repetir la validacion sin depender de observacion manual.

6.1 Validacion funcional de la escena

La validacion principal de `Sim_T2_Phase1_Basic.ttt` termino con `passed=true`. Los indicadores mas relevantes fueron:

Indicador	Resultado
Estado final de tarea	CHARGING
Modo final de movimiento	ARRIVED_TARGET
Tareas completadas	4/4
Distancia caminada por Bill	7,854 m
Bateria final R1	71,88 %
Sensores activos	16
Rayos de sensor visualizados	16
Landmarks/celdas SLAM finales	22 en validacion principal
Actualizaciones SLAM	4822
Error final de pose	0,031 m
Error maximo de pose	0,044 m
Distancia minima positiva a obstaculo	0,079 m
Riesgo maximo de obstaculo	0,762

Tambien se verificaron condiciones cualitativas: Bill se mueve entre WS1 y WS2, Bill ejecuta acciones visuales de tomar/trabajar/entregar, los shelves y mesas estan presentes, las piezas T1/T2 tienen geometria visual no generica, la cola de tareas se publica, el robot vuelve a carga y el planner SLAM activa `SLAM_WAYPOINT` al detectar bloqueo local.

6.2 Desempeno del estimador Kalman-landmark

En la exportacion especifica del estimador, la escena usa PID como controlador base y KALMAN_LANDMARK como algoritmo SLAM. Los resultados fueron:

- RMSE de posicion: 0,031.25 m.
- Error medio de posicion: 0,029.58 m.
- Error P95: 0,047.10 m.
- Error maximo: 0,062.72 m.
- RMSE de orientacion: 0,018.09 rad.
- Covarianza media publicada: 0,00081.
- Landmarks finales: 16.
- Actualizaciones Kalman/SLAM: 2615.

Phase 1 Basic - desempeno del estimador

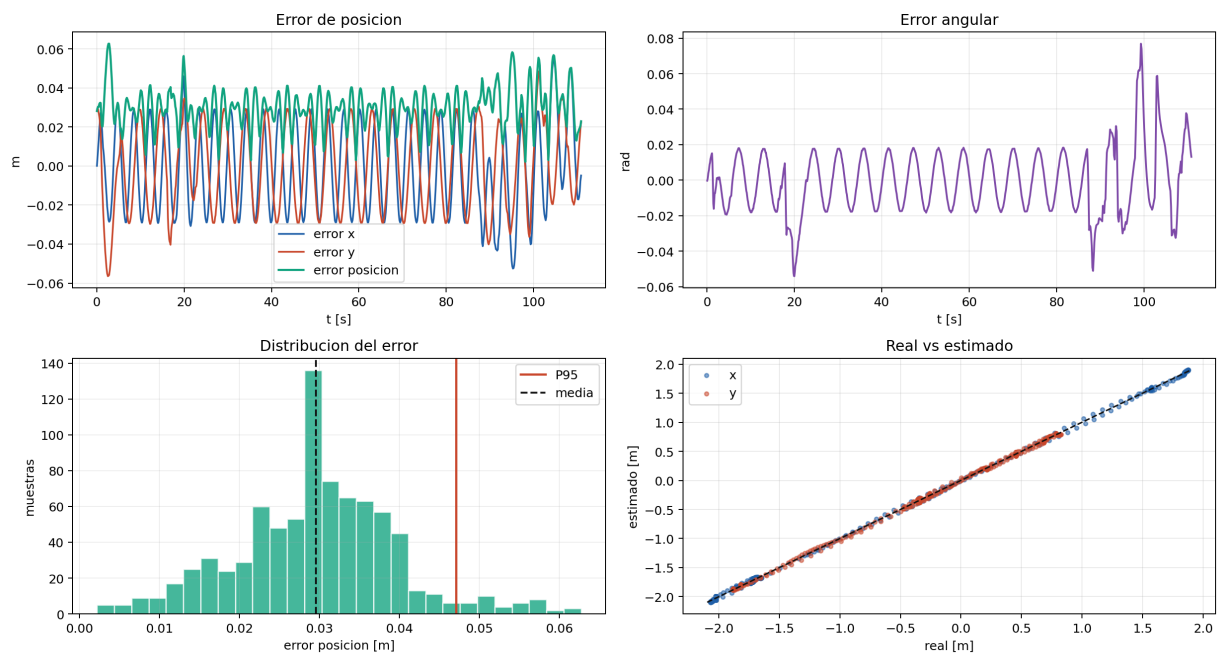


Figura 6.1: Desempeno del estimador Kalman-landmark: error de pose, covarianza publicada, landmarks y actualizaciones.

6.3 Comparacion de algoritmos SLAM

La comparacion se ejecuto con el mismo escenario, sensores, tareas y controlador para los cuatro modos de SLAM. La Tabla 6.2 resume los valores principales.

Cuadro 6.2: Benchmark SLAM reducido en CoppeliaSim.

Metrica	GMapping	Hector	Cartographer	Kalman
Completado	Si	Si	Si	Si
Duracion [s]	102,60	102,50	101,65	101,60
Ruta [m]	10,906	10,880	10,774	10,771
RMSE pos. [m]	0,04135	0,04072	0,03842	0,03160
Error P95 [m]	0,05697	0,06485	0,05673	0,04959
Error max. [m]	0,07722	0,11112	0,09340	0,06433
Features finales	63	56	55	15
Updates SLAM	1457	1426	1432	1447
Submaps	0	0	10	0
Loop closures	0	0	1	0
Bateria usada [%]	16,82	16,78	16,66	16,63

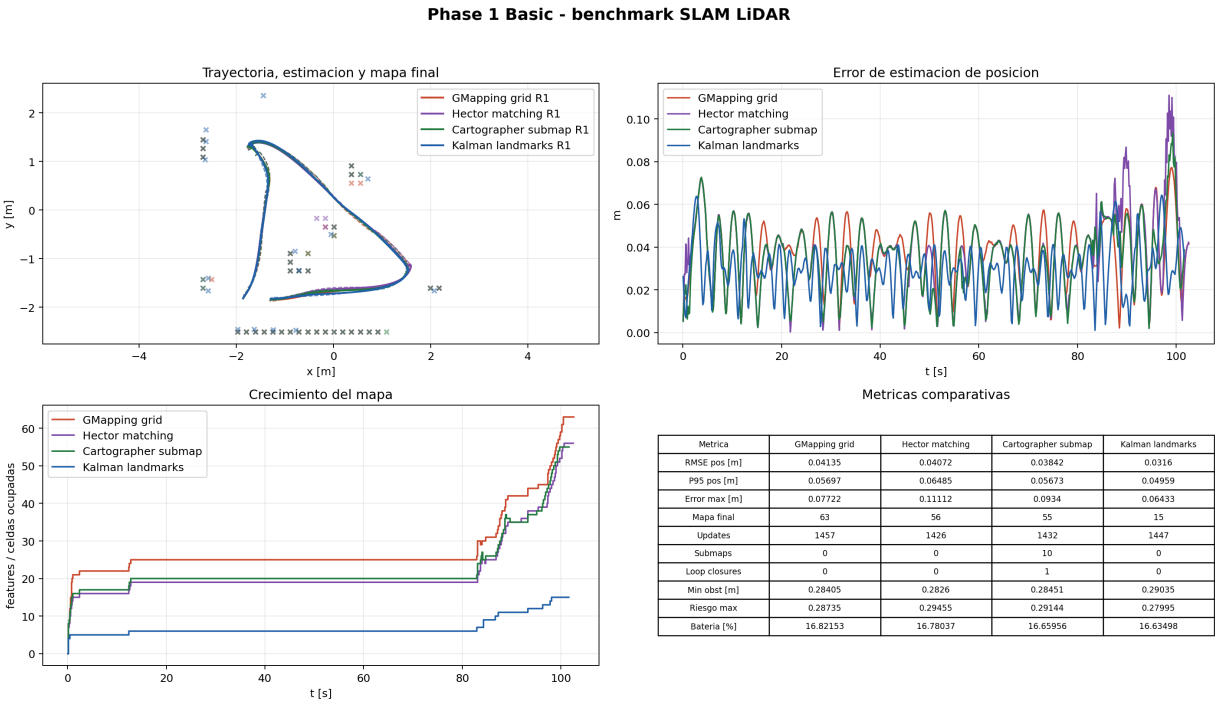


Figura 6.2: Comparacion grafica de los cuatro modos SLAM reducidos: precision, mapa, actualizaciones y bateria.

6.4 Lectura de resultados

Kalman-landmark fue el mejor en error de pose y consumo, porque el escenario es pequeno y las landmarks son suficientes para navegar. GMapping y Hector generan mapas mas densos, utiles para justificar grilla de ocupacion y scan matching, pero pagan un pequeno incremento en error y complejidad. Cartographer-submap consigue un equilibrio interesante: menor ruta que GMapping/Hector, estructura por submaps y un cierre de ciclo simplificado, aunque no supera el RMSE de Kalman-landmark en esta celda compacta.

La conclusion tecnica es que para esta fase basica conviene defender KALMAN_LANDMARK como estimador principal y usar GMapping/Hector/ Cartographer como comparacion academica de mapas LiDAR reducidos.

7

Validacion y trazabilidad

La validacion se diseno para demostrar comportamiento, no solo compilacion. Se evalua si la escena existe, si los objetos clave estan presentes, si los scripts estan embebidos, si las tareas se completan, si Bill se mueve, si las acciones visuales ocurren, si el sistema anticollision se activa, si el estimador queda acotado y si los sensores publican rayos visibles.

7.1 Checklist automatizado

Bloque	Chequeo	Estado
Escena	Existe Sim_T2_Phase1_Basic.ttt y contiene R1, B1, T1, T2, C1.	OK
Warehouse	Dos mesas, dos shelves, obstaculos y panel de tareas.	OK
Scripts	Controlador R1 y manager B1 adjuntos en la escena.	OK
Tareas	Entrega y retorno de T1/T2; cuatro tareas completadas.	OK
Bill	Movimiento entre estaciones y acciones de tomar/trabajar/entregar.	OK
Percepcion	16 sensores activos y rayos visualizados.	OK
Seguridad	Modo AVOIDING observado y riesgo de obstaculo publicado.	OK
Estimacion	Kalman activo, mapa SLAM actualizado y error acotado.	OK
Planificacion	SLAM_WAYPOINT observado.	OK
Bateria	Tarea aceptada por bateria y retorno a carga.	OK

7.2 Timeline de la mision

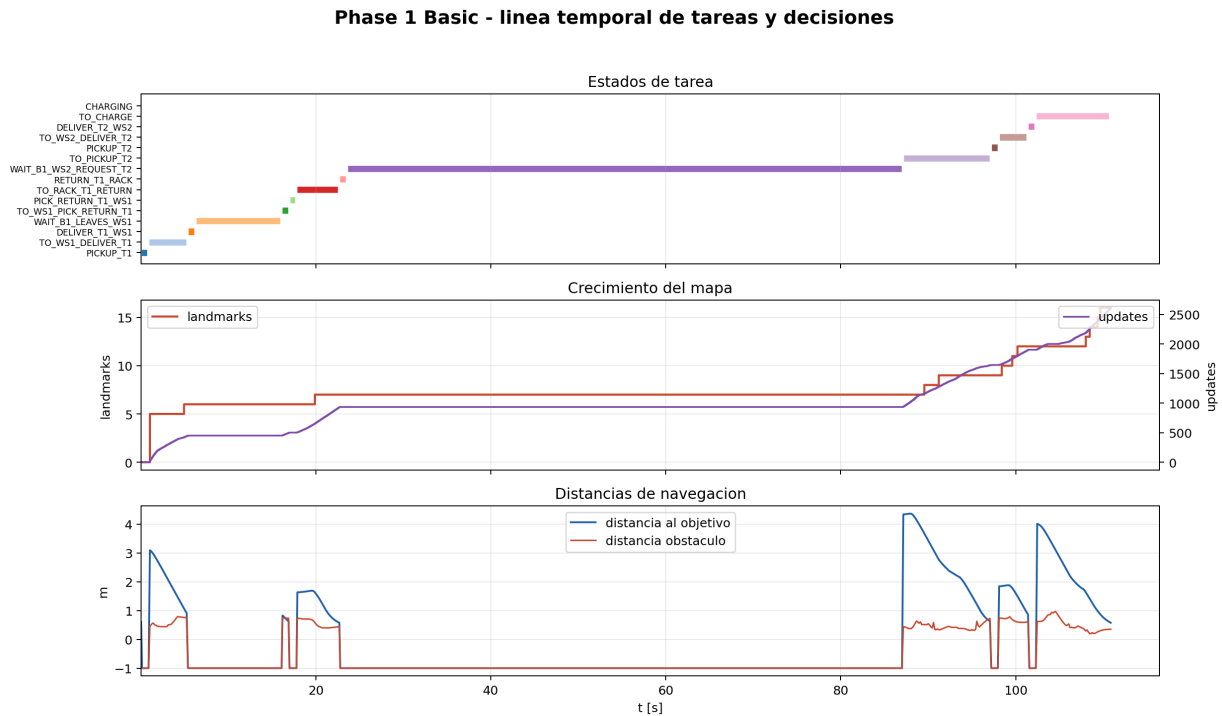


Figura 7.1: Timeline exportado de la simulacion: estados de tarea, movimiento, planner y acciones de Bill durante la mision.

La mision no es instantanea porque Bill introduce esperas reales de proceso. R1 debe detenerse mientras Bill trabaja T1 o T2 y debe esperar a que B1 llegue a WS2 antes de buscar T2. Esta espera se refleja en el modo WAIT_B1.

7.3 Senales exportadas

Las senales principales usadas para logs y plots son:

- Tarea: phase1TaskState, phase1TaskId, phase1CompletedTaskCount.
- Movimiento: phase1MotionMode, phase1PlannerMode, phase1DistanceToTarget.
- Seguridad: phase1MinObstacleDistance, phase1ObstacleRisk, phase1SensorRayCount.
- Estimacion: phase1EstimatedPose, phase1PoseError, phase1KalmanCovariance.
- Mapeo: phase1SlamLandmarkCount, phase1SlamUpdates, phase1GridOccupiedCells.
- Bill: phase1B1Station, phase1B1Action, phase1B1MovedDistance.
- Energia: phase1BatteryLevel, phase1BatteryMode, phase1Charging.

7.4 Reproducibilidad

El flujo reproducible queda:

1. Construir y validar escena: `pythonactividad2/coppeliassim/build_phase1_basic_scene.py`.
2. Exportar corrida base: `pythonactividad2/coppeliassim/run_phase1_slam_export.py`.
3. Comparar SLAM: `pythonactividad2/coppeliassim/run_phase1_slam_comparison.py`.
4. Generar plots: `pythonactividad2/coppeliassim/plot_phase1_slam_comparison.py`.
5. Compilar informe: `pdflatex->bibtex->pdflatex->pdflatex`.

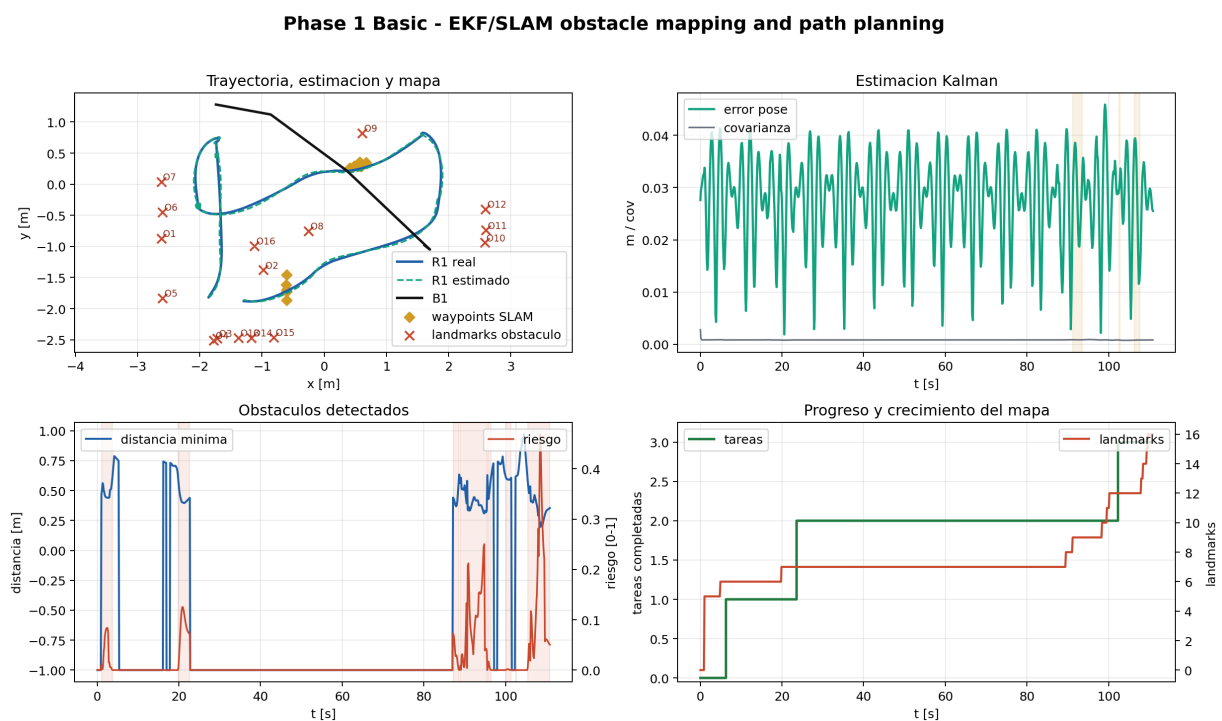


Figura 7.2: Resumen visual de la corrida base: pose, error, bateria, obstaculos, planner y estados principales.

8

Conclusiones

La simulacion cumple el objetivo de la Actividad 2 y extiende el ejercicio base de Pioneer hacia una celda de trabajo mas realista. R1 no solo se desplaza hacia un objetivo: interpreta una cola de tareas, coordina entregas con Bill, transporta T1/T2, espera procesos humanos, devuelve piezas al shelf correcto, evita obstaculos, estima su pose, actualiza un mapa y termina en la estacion de carga.

El filtro Kalman-landmark es el metodo mas conveniente para defender la version basica del escenario. Es simple, reproducible, presenta el menor RMSE de posicion y genera un mapa compacto suficiente para planificacion local. Las variantes de grilla tipo GMapping, Hector y Cartographer enriquecen el benchmark porque muestran enfoques populares de SLAM LiDAR: ocupacion por log-odds, scan matching y submaps con loop closure. Sin embargo, deben presentarse como implementaciones reducidas en CoppeliaSim, no como ejecucion de paquetes ROS.

El sistema anticolisiones funciona como una capa reactiva clara: cuando los sensores detectan obstaculos cercanos, el robot reduce velocidad, modifica el giro y publica riesgo. La capa de waypoint SLAM mejora este comportamiento al introducir desvios locales cuando el mapa detecta bloqueo entre R1 y el objetivo.

Respecto a control, la comparacion P/PI/PID/LQR/NMPC se mantiene como analisis complementario. PI obtuvo el mejor score ponderado de los logs disponibles, pero la defensa academica principal debe centrarse en los metodos clasicos efectivamente trazables: cinematica diferencial, control proporcional/PID, evitacion reactiva, Kalman y mapeo de ocupacion/landmarks.

Como trabajo futuro, el escenario puede escalarse a B1/B2/B3 y R1/R2/R3 con un asignador de tareas por cola y reservas temporales. Tambien podria conectarse a ROS 2 para ejecutar paquetes reales de `slam_toolbox`, Cartographer o Hector/GMapping equivalentes, comparar contra ground truth y separar la capa de simulacion de la capa de algoritmo.

Bibliografía

- [1] R. E. Kalman, "A new approach to linear filtering and prediction problems," *Journal of Basic Engineering*, vol. 82, no. 1, pp. 35–45, 1960.
- [2] G. Grisetti, C. Stachniss, and W. Burgard, "Improved techniques for grid mapping with rao-blackwellized particle filters," *IEEE Transactions on Robotics*, vol. 23, no. 1, pp. 34–46, 2007.
- [3] S. Kohlbrecher, O. von Stryk, J. Meyer, and U. Klingauf, "A flexible and scalable slam system with full 3d motion estimation," in *2011 IEEE International Symposium on Safety, Security, and Rescue Robotics*, pp. 155–160, 2011.
- [4] W. Hess, D. Kohler, H. Rapp, and D. Andor, "Real-time loop closure in 2d lidar slam," in *2016 IEEE International Conference on Robotics and Automation*, pp. 1271–1278, 2016.



Anexos

A.1 Archivos principales

Archivo	Funcion
coppeliassim/Sim_T2_Phase1_Basic.ttt	Escena final validada.
coppeliassim/Sim_T2_Phase1_Basic_validation.json	Resultado de validacion automatizada.
coppeliassim/build_phase1_basic_scene.py	Generador de escena y validador headless.
coppeliassim/phase1_r1_task_controller.lua	Controlador principal de R1: tareas, navegacion, SLAM, bateria y telemetria.
coppeliassim/phase1_b1_walking_manager.lua	Movimiento y acciones visuales de Bill.
coppeliassim/run_phase1_slam_export.py	Exportacion CSV de corrida base.
coppeliassim/run_phase1_slam_comparison.py	Benchmark de algoritmos SLAM reducidos.
coppeliassim/plot_phase1_slam_comparison.py	Generacion de plots de comparacion SLAM.
coppeliassim/phase1_slam_logs/	CSV, JSON, Markdown y figuras del benchmark SLAM.
coppeliassim/phase1_controller_logs/	Logs y plots de comparacion de controladores.

A.2 Parametros relevantes

Parametro	Valor	Comentario
Radio de rueda	0,097.5 m	Conversion de velocidad lineal a angular de motor.
Distancia entre ruedas	0,331 m	Modelo diferencial del Pioneer P3DX.
Velocidad maxima	0,55 m/s	Saturacion de avance.
Velocidad angular maxima	1,32 rad/s	Saturacion de giro.
Rango de evitacion	0,62 m	Umbral de influencia reactiva de obstaculos.
Hard stop	0,12 m	Distancia critica frontal.
Bateria inicial	96 %	Valor usado en validaciones.
Umbral bateria baja	56 %	Limita velocidad si se alcanza.
Umbral bateria critica	26 %	Modo conservador.
Resolucion de grilla	0,18 m	Usada por GMapping/Hector/Cartographer reducidos.
Clearance de planner	0,40 m	Separacion minima deseada respecto al mapa.
Offset de desvio	0,52 m	Distancia lateral del waypoint local.

A.3 Resumen de benchmark de control

Modo	Dur. [s]	Mov. [s]	Ruta [m]	Bat. [%]	Riesgo max.	Score
P	115,20	37,70	12,843	19,804	0,410	0,573
PI	110,30	31,20	12,867	19,516	0,487	0,274
PID	110,85	31,55	12,856	19,784	0,469	0,567
LQR	114,15	36,20	12,796	19,505	0,401	0,436
NMPC	112,30	33,05	12,788	19,316	0,444	0,326

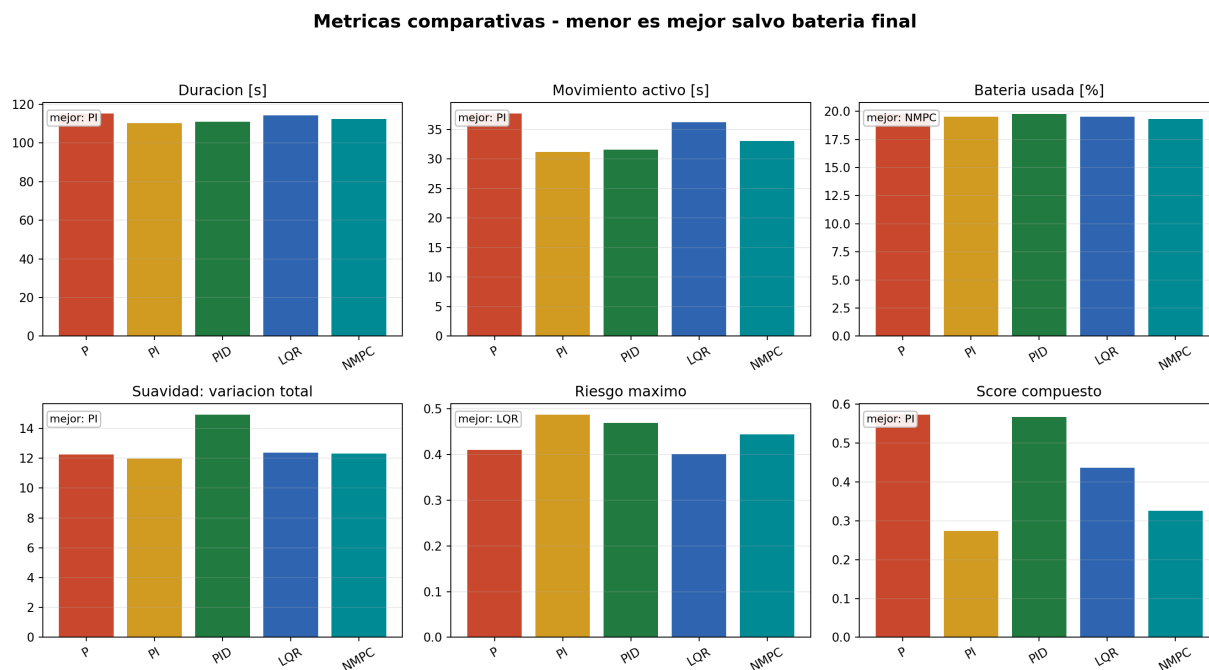


Figura A.1: Métricas complementarias de control: tiempos, trayectoria, suavidad, batería, riesgo y error de pose.

A.4 Notas de limitacion

- No se afirma ejecución de `slam_gmapping`, Hector ROS ni Google Cartographer real; las variantes son reproducciones reducidas dentro de CoppeliaSim.
- El sensor se representa con los 16 ultrasonidos del Pioneer visualizados como rayos tipo LiDAR 2D, suficiente para la actividad y para comparar mapas.
- La planificación local por waypoint no reemplaza a A* global, D* o RRT; es una regla práctica para evitar bloqueos en una celda pequeña.
- Los modos LQR/NMPC del benchmark de control se conservan como aproximaciones comparativas; la defensa principal se basa en los métodos clásicos trazables y validados.